AI ENGINEERING / WEEK 2 / MODULE 02

Saturday Quick Reference

NumPy • list vs array • numerical computing

10 topic recaps • essential methods • runnable code • common pitfalls

WHAT YOU WILL REVISE

01 Python list vs NumPy: runtime

02 Arrays, shape and data types

03 Dimensions and array initialization

04 Indexing, slicing and reshape

05 Flatten, transpose and combine

06 Vectorized math, dot and matrix multiplication

07 Statistics and axis directions

08 Boolean masks and conditional selection

09 Broadcasting on compatible shapes

10 NaN, infinity, storage and ML readiness

HOW TO USE THIS REFERENCE

Read the definition, learn what each method does, run the short code in Google Colab, then attempt the two exercises supplied after each topic in the presentation.

40 core Python demonstrations in the Saturday/Sunday slide deck; examples here are condensed for revision.

01 Python list vs NumPy: runtime

A Python list is a flexible collection. A NumPy ndarray is a numerical array that applies arithmetic to whole arrays. Time the same task to compare runtime; measured values depend on the Colab runtime.

IMPORTANT METHODS / WHAT THEY DO

list(range(n))	Build a list of n whole numbers.
np.array(values)	Convert numbers into a NumPy array.
time.time()	Read clock time; subtract to measure elapsed seconds.
np.allclose(a,b)	Check that numeric results are sufficiently close.

COLAB CODE / EACH STEP COMMENTED

```
import time  # Read clock time in seconds.
import numpy as np  # Load numerical arrays.
numbers = list(range(100000))  # Make test values.
arr = np.array(numbers)  # Make equivalent array.
start = time.time()  # Start list timer.
list_out = [x + 5 for x in numbers]  # Add five.
list_sec = time.time() - start  # Elapsed time.
start = time.time()  # Start NumPy timer.
array_out = arr + 5  # Vectorized addition.
array_sec = time.time() - start  # Elapsed time.
print(list_sec, array_sec)  # Compare seconds.
```

REMEMBER: Benchmark tip: use identical input sizes and operations; check outputs before interpreting speed.

02 Arrays, shape and data types

An ndarray usually stores values of one data type. Its shape shows how many items exist along each dimension; its dtype shows the type used for those values.

IMPORTANT METHODS / WHAT THEY DO

np.array([...])	Create an array.
arr.shape / arr.ndim	See dimension lengths / number of axes.
arr.size / arr.dtype	See element count / data type.
arr.astype(float)	Return values converted to float.
np.arange(start,stop,step)	Create a regularly spaced sequence.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Use NumPy.
marks = np.array([70, 80, 90])  # Numeric data.
print(marks.shape)  # (3,): three values.
print(marks.ndim)  # 1: one dimension.
print(marks.size)  # 3: element count.
print(marks.dtype)  # Stored numeric type.
print(marks.astype(float))  # Convert to floats.
print(np.arange(0, 10, 2))  # 0,2,4,6,8.
```

REMEMBER: Remember: shape (3,) is 1D, while shape (3, 1) is 2D.

03 Dimensions and array initialization

A 2D array represents rows and columns. Initializer functions quickly build matrices of zeros, ones, fixed values, and evenly spaced numbers.

IMPORTANT METHODS / WHAT THEY DO

np.zeros((r,c))	Create a matrix of zeros.
np.ones((r,c))	Create a matrix of ones.
np.full((r,c), v)	Fill every element with v.
np.eye(n)	Create an identity matrix.
np.linspace(a,b,n)	Create n evenly spaced values.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
features = np.array([[2, 70], [4, 90]])
print(features.shape)  # (2, 2): rows, columns.
print(np.zeros((2, 3)))  # Two rows, three columns.
print(np.ones((2, 2)))  # All ones.
print(np.full((2, 2), 7))  # All sevens.
print(np.eye(3))  # Identity matrix.
print(np.linspace(0, 1, 5))  # Five points.
```

REMEMBER: AI use: each row can represent a learner; columns can represent their numerical features.

04 Indexing, slicing and reshape

Indexing retrieves one position; slicing retrieves a range. reshape() changes the view of dimensions without changing the number of elements.

IMPORTANT METHODS / WHAT THEY DO

a[0]	Select first element (index starts at 0).
a[1:4]	Select indexes 1, 2 and 3.
a[row,col]	Select element at a row and column.
a[:,1]	Select second column of every row.
a.reshape(r,c)	Rearrange into compatible dimensions.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
a = np.array([10, 20, 30, 40, 50, 60])
print(a[0])  # First value: 10.
print(a[1:4])  # Values 20, 30, 40.
mat = a.reshape(2, 3)  # 2 rows x 3 columns.
print(mat[1, 0])  # Row 1, column 0: 40.
print(mat[:, 1])  # Second column: [20,50].
```

REMEMBER: reshape() must preserve total element count: 6 elements can become (2,3), not (4,2).

05 Flatten, transpose and combine

Reshape changes dimensions; flatten/ravel make one dimension; transpose swaps axes. Stacking joins arrays in rows or columns.

IMPORTANT METHODS / WHAT THEY DO

reshape()	Change shape but keep element count.
flatten() / ravel()	Return a 1D result (copy/view behavior differs).
a.T	Transpose the axes.
np.vstack((a,b))	Stack two compatible arrays vertically.
np.hstack((a,b))	Join compatible arrays horizontally.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
a = np.array([[1, 2], [3, 4]])
print(a.flatten())  # One-dimensional copy.
print(a.T)  # Swap rows and columns.
print(np.vstack((a, a)))  # Add more rows.
print(np.hstack((a, a)))  # Add columns.
print(np.split(np.arange(6), 3))  # 3 parts.
```

REMEMBER: Remember: flatten() returns a copy; ravel() may return a view depending on layout.

06 Vectorized math, dot and matrix multiplication

Vectorized arithmetic processes arrays element by element. Dot products and matrix multiplication combine feature and weight values used in ML models.

IMPORTANT METHODS / WHAT THEY DO

a+b / a*b	Elementwise addition / multiplication.
np.square(a) / np.sqrt(a)	Square elements / take square roots.
np.dot(a,b) / a @ b	Calculate vector dot or matrix product.
np.sum(a)	Add array values.
np.clip(a,low,high)	Limit values to a selected range.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + 5)  # Add to every element.
print(a * b)  # Multiply matching elements.
print(np.dot(a, b))  # Weighted sum: 32.
print(np.square(a))  # [1,4,9].
print(np.clip(b, 0, 5))  # Maximum becomes 5.
```

REMEMBER: Important: * is elementwise; @ represents matrix multiplication for compatible arrays.

07 Statistics and axis directions

Statistics summarize data: mean/median describe typical values; standard deviation describes spread. axis=0 summarizes down rows (per column), axis=1 across columns (per row).

IMPORTANT METHODS / WHAT THEY DO

np.sum(a,axis=...)	Sum all values or by axis.
np.mean(a) / np.median(a)	Average / middle value.
np.std(a)	Standard deviation.
np.min(a) / np.max(a)	Smallest / largest value.
axis=0 / axis=1	Per-column / per-row aggregation.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
marks = np.array([[70, 80], [90, 100]])
print(np.mean(marks))  # Overall average: 85.
print(np.mean(marks, axis=0))  # [80,90].
print(np.mean(marks, axis=1))  # [75,95].
print(np.median(marks))  # Middle: 85.
print(np.std(marks))  # Overall spread.
```

REMEMBER: Shape is (students, subjects) here: axis=0 = subject averages; axis=1 = student averages.

08 Boolean masks and conditional selection

Array comparisons create Boolean (True/False) values. Masks extract matching values and np.where() chooses an output for each condition.

IMPORTANT METHODS / WHAT THEY DO

a >= threshold	Produce a True/False mask.
a[mask]	Select only matching elements.
np.where(mask,x,y)	Return x when True, otherwise y.
np.any(mask) / np.all(mask)	Are any / all values True?
np.array_equal(a,b)	Check exact array equality.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
marks = np.array([45, 65, 80])
passed = marks >= 50  # Boolean mask.
print(passed)  # [False, True, True].
print(marks[passed])  # Select passing marks.
print(np.where(passed, 'Pass', 'Fail'))
print(np.all(passed))  # False.
```

REMEMBER: A mask must be compatible with the shape of the array being indexed.

09 Broadcasting on compatible shapes

Broadcasting applies operations between different, compatible array shapes without explicitly copying the smaller array.

IMPORTANT METHODS / WHAT THEY DO

array + scalar	Add one value to every element.
matrix + row_vector	Add per-column values to every row.
arr.shape	Inspect each operand shape.
np.newaxis	Insert an axis of length one.
np.broadcast_shapes()	Check whether shapes are compatible.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
x = np.array([[10, 20], [30, 40]])
bonus = np.array([5, 10])  # Per-column bonus.
print(x + bonus)  # Add bonus to each row.
print(x.shape)  # (2,2).
print(bonus.shape)  # (2,).
print(np.broadcast_shapes(x.shape, bonus.shape))
```

REMEMBER: Rule: compare shapes from the right; each pair must match or one dimension must be 1.

10 NaN, infinity, storage and ML readiness

Missing or infinite numerical values can break learning algorithms. Detect and handle them, verify the feature matrix, and save useful arrays.

IMPORTANT METHODS / WHAT THEY DO

np.isnan(x) / np.isfinite(x)	Detect NaN / check finite values.
np.nan_to_num(x)	Replace NaN and infinities with numbers.
np.save() / np.load()	Save and restore an .npy array.
np.random.default_rng(seed)	Make a reproducible random generator.
x.ndim / x.shape	Check feature-matrix dimensions.

COLAB CODE / EACH STEP COMMENTED

```
import numpy as np  # Load NumPy.
x = np.array([1.0, np.nan, np.inf])
print(np.isfinite(x))  # True, False, False.
clean = np.nan_to_num(x, nan=-1, posinf=0)
print(clean)  # Missing/infinite replaced.
np.save('clean.npy', clean)  # Save to file.
loaded = np.load('clean.npy')  # Read file.
print(loaded)  # Verify restored array.
```

REMEMBER: Before training: expect 2D X (samples x features), correct dtype and finite values. Colab runtime files are temporary.

FAST LOOKUP / CORE FUNCTIONS

Array basics
np.array, np.arange, np.linspace, .shape, .ndim, .size, .dtype, .astype

Initialize
np.zeros, np.ones, np.full, np.eye

Index + shape
a[0], a[1:4], a[:,1], reshape, flatten, ravel, .T

Combine
np.vstack, np.hstack, np.split

Math + stats
a+b, a*b, np.dot, @, square, sqrt, clip, sum, mean, median, std, min, max

Filter + broadcast
a>0, a[mask], np.where, np.any, np.all, np.broadcast_shapes

Numerical quality
np.isnan, np.isfinite, np.nan_to_num, np.array_equal, np.allclose

Save + reproducibility
np.save, np.load, np.random.default_rng

EXAM / LAB REMINDERS

- NumPy arrays work most naturally with numerical values of consistent dtype.
- axis=0 works down rows (per column); axis=1 works across columns (per row).
- Always validate shape and missing / infinite numbers before model training.
- Use time.time() measurements for demonstration, not as an absolute speed guarantee.